

Applying recurrences for  
time complexity analysis

# Mergesort

- Suppose that a function `merge(L1, L2)` which accepts two sorted linked lists, `L1` and `L2`, merges them into one sorted linked list, `L`, and outputs `L`.
- `merge(L1, L2)` needs  $O(|L1|+|L2|)$  time.

# Mergesort(L)

- If  $|L|=0$  or 1, return L.   $c_1$  time
- Divide the linked list into two halves, L1 and L2.   $c_2n$  time
- Call Mergesort(L1);  ?
- Call Mergesort(L2);  ?
- $L = \text{Merge}(L1, L2)$ ;   $c_3n$  time
- Return L;   $c_4$  time

Let  $T(n)$  be the running time for  
Mergesort( $L$ ),  $n=|L|$

$T(n)$  must be a monotonic increasing function of  $n$ .

- If  $|L|=0$  or  $1$ , return  $L$ . —  $c_1$  time
  - Divide the linked list into two halves,  $L1$  and  $L2$ . —  $c_2n$  time
  - Call Mergesort( $L1$ ); —  $T(\lceil n/2 \rceil)$
  - Call Mergesort( $L2$ ); —  $T(\lfloor n/2 \rfloor)$
  - $L = \text{Merge}(L1, L2)$ ; —  $c_3n$  time
  - Return  $L$ ; —  $c_4$  time
- $L1$  and  $L2$  differ at most  $1$ .

Let  $T(n)$  be the running time for  
Mergesort( $L$ ),  $n=|L|$

Summing up,  $T(n)=T(\lceil n/2 \rceil)+T(\lfloor n/2 \rfloor)+c_1+c_2n+c_3n+c_4$

- If  $|L|=0$  or  $1$ , return  $L$ . —  $c_1$  time
  - Divide the linked list into two halves,  $L1$  and  $L2$ . —  $c_2n$  time
  - Call Mergesort( $L1$ ); —  $T(\lceil n/2 \rceil)$
  - Call Mergesort( $L2$ ); —  $T(\lfloor n/2 \rfloor)$
  - $L=\text{Merge}(L1,L2)$ ; —  $c_3n$  time
  - Return  $L$ ; —  $c_4$  time
-   $L1$  and  $L2$  differ at most  $1$ .

Let  $T(n)$  be the running time for  
Mergesort( $L$ ),  $n=|L|$

- $T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + c_1 + c_2n + c_3n + c_4$
- $\Rightarrow$
- $T(n) \leq 2T(\lceil n/2 \rceil) + cn$ , where  $c = c_1 + c_2 + c_3 + c_4$
- $T(n) \geq 2T(\lfloor n/2 \rfloor) + c'n$ , where  $c' = c_2$
- Depending on upper or lower bounds we want to find.

# Solving Recurrences

- $T(n) \leq 2T(\lceil n/2 \rceil) + cn$
- $T(1)=1$ . Or  $T(1)=k \geq 1$ , a constant. Don't care!
- Base cases are understood, so we need not mention them.
- We can always have  $T(1), T(2), \dots T(10), \dots$ , as many base cases as we want. But there must be constant number of base cases.

# Simplify the Recurrences first

- In most of the time, we aim at big-O or big-Omega notations. So, we remove the noises.
- For solving  $T(n) \leq 2T(\lceil n/2 \rceil) + cn$
- Just consider  $T(n) = 2T(n/2) + n$
- Why possible?



$$T(n) \leq 2T(\lceil n/2 \rceil) + cn$$

- Case 1: when  $n$  is a power of 2.
- The recurrence becomes:
- $T(n) \leq 2T(n/2) + cn$
- $\leq 2[2T(n/4) + cn/2] + cn$
- $= 4T(n/4) + 2cn$
- $\leq 4[2T(n/8) + cn/4] + 2cn$
- $= 8T(n/8) + 3cn$
- ...
- $\leq 2^k T(n/2^k) + kcn$ , where  $2^k = n \rightarrow k = \log n$
- $< nT(1) + cn \log n$
- $= O(n \log n)$

$$T(n) \leq 2T(\lceil n/2 \rceil) + cn$$

- Case 1: when  $n$  is a power of 2.
- The recurrence becomes:
- $T(n) \leq 2T(n/2) + cn$
- $\leq 2[2T(n/4) + cn/2] + cn$
- $= 4T(n/4) + 2cn$
- $\leq 4[2T(n/8) + cn/4] + 2cn$
- $= 8T(n/8) + 3cn$
- ...
- $\leq 2^k T(n/2^k) + kcn$ , where  $2^k = n \rightarrow k = \log n$
- $< nT(1) + cn \log n$
- $= O(n \log n)$

For  $n$  is a power of 2,  $c$  will be absorbed by big-O.  
Changing “ $\leq$ ” to “ $=$ ” does not change the final expression  $O(n \log n)$ .

$$T(n) \leq 2T(\lceil n/2 \rceil) + cn$$

- Case 1: when  $n$  is NOT a power of 2.
- Then,  $\exists m \bullet 2^{m-1} < n < 2^m < 2n$ .
- Since  $T$  is a monotonic increasing function,
- $T(n) \leq T(2^m)$
- $< 2^m T(1) + c 2^m \log 2^m$
- $< 2n T(1) + c (2n) \log (2n)$
- $= 2n T(1) + 2c n \log n + 2cn$
- $= O(n \log n)$

For  $n$  is NOT a power of 2, changing “ $\leq$ ” to “ $=$ ”, removing  $c$ , and ignoring the floor function, do not affect the final expression  $O(n \log n)$ .

# Methods for solving recurrences

- Called expansion
- Called recursive method in MIT textbook.
- $T(n) \leq 2T(n/2) + cn$
- $\leq 2[2T(n/4) + cn/2] + cn$
- $= 4T(n/4) + 2cn$
- $\leq 4[2T(n/8) + cn/4] + 2cn$
- $= 8T(n/8) + 3cn$
- ...
- $\leq 2^k T(n/2^k) + kcn$ , where  $2^k = n \rightarrow k = \log n$
- $< nT(1) + cn \log n$
- $= O(n \log n)$

# Methods for solving recurrences

- Another method:
- Called Mathematical induction
- Called substitution in MIT textbook.
- $T(n) = 2T(n/2) + cn$
- Claim:  $T(n) < c n \log n$  for some  $c > 1$
- For this method, NEVER use big-O or  $\Omega$ .
- Base case:  $n=2$ .
- $T(2) = 1 < 2c$  ( $T(1) = 1$  is understood, though not given)
- Assume  $T(n) < c n \log n$ , for  $2 \leq n < m$ .
- Consider  $T(m)$ .
- $T(m) = 2T(m/2) + cm < 2c(m/2)\log(m/2) + cm$
- $= cm \log m - cm + cm = cm \log m$ .
- Result from induction.

# Methods for solving recurrences

- Another method:
- Called Mathematical induction
- Called substitution in MIT textbook.

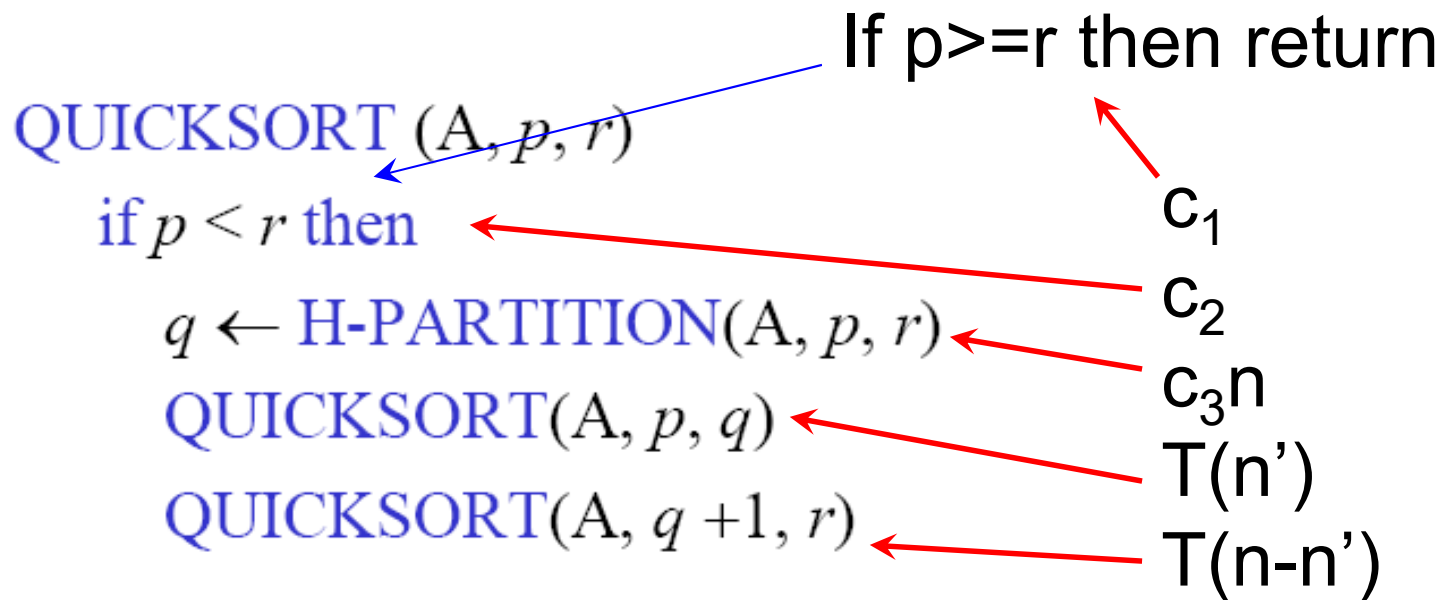
- $T(n) = 2T(n/2) + cn$
- Claim:  $T(n) < c n \log n$  for some  $c > 1$
- For this method, NEVER use big-O or  $\Omega$ .

- ~~• Base case:  $n=2$ .~~
- ~~•  $T(2)=1 < 2c$  ( $T(1)=1$  is understood, though not given)~~
  - Assume  $T(n) < c n \log n$ , for  $2 \leq n < m$ .
  - Consider  $T(m)$ .
- $T(m) = 2T(m/2) + cm < 2c(m/2)\log(m/2) + cm$
- $= cm \log m - cm + cm = cm \log m$ .
- ~~• Result from induction.~~

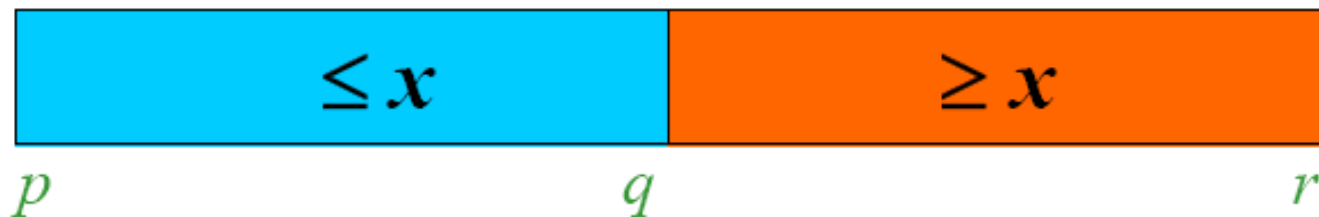
# Methods for solving recurrences

- Called Mathematical induction
- Called substitution in MIT textbook.
- Simply it further.
- No need to use m.
- $T(n) = 2T(n/2) + cn$
- Claim:  $T(n) < c n \log n$  for some  $c > 1$
- Assume  $T(n) < c n \log n$ , for  $n \geq 2$ .
- Consider  $T(n)$ .
- $T(n) = 2T(n/2) + cn < 2c(n/2)\log(n/2) + cn$
- $= cn \log n - cn + cn = cn \log n$ .
- What have we done:  $\exists c=2, n_0=2 \bullet \forall n > n_0 \bullet T(n) < cn \log n$ .

# Quicksort --- $T(n)$



$$T(n) = T(n') + T(n - n') + n$$





# Worst case time complexity

- Let  $T(n)$  be the worst case time of quicksort with  $n$  input elements.
- $T(n) = n +$
- $\max(T(1)+T(n-1), T(2)+T(n-2), \dots, T(n-1)+T(1)).$
- Claim:  $T(n) < c n^2$ ,  $c \geq 2$ .
- Suppose  $T(m)+T(n-m)$  is the maximum, for some  $m$ , where  $1 \leq m \leq n-1$ .
- $T(n) = n + T(m)+T(n-m) < n + c m^2 + c (n-m)^2$

- $T(n) < n + c m^2 + c (n-m)^2$
- $< c (n-m)^2 + c m^2 + 2cm(n-m)$
- $= c [(n-m)^2 + 2m(n-m) + m^2]$
- $= c (n-m + m)^2$
- $= c n^2.$
- $2cm(n-m) - n$  is minimum when  $m=1$  or  $n-1$ .
- $2cm(n-m) - n \geq$
- $2c(n-1) - n \geq$
- $4(n-1) - n = 3n-4$
- $> 0$
- Choose  $n \geq 2$ .

What has been done:

$$\exists c, n_0=2, \forall n > n_0, T(n) < cn^2$$

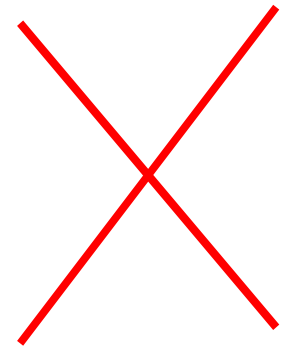
- $O(n^2)$  is tight.
- Worst case scenario.
- Always have one partition with one element.

# Average case time complexity

- Let  $T(n)$  be the average case time of quicksort with  $n$  input elements.
- $T(n) = n +$
- $(1/(n-1))(T(1)+T(n-1)) + (1/(n-1))(T(2)+T(n-2)) + \dots + (1/(n-1))(T(n-1)+T(1)).$
- $= n + (2/(n-1))(T(1)+T(2)+\dots T(n-1))$
- $= n + (2/(n-1)) \sum_{1 \text{ to } n-1} T(i)$
- Claim:  $T(n) < c n \log n$ ,  $c \geq 4$ .
- $T(n) < n + (2c/(n-1)) \sum_{1 \text{ to } n-1} (i \log i)$

# A failure ?!

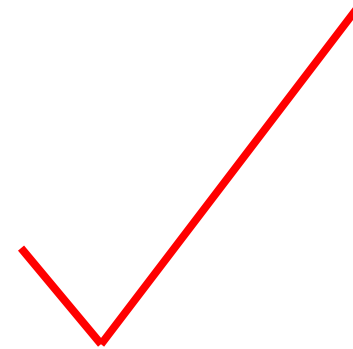
- Claim:  $T(n) < c n \log n$ ,  $c \geq 4$ .
- $T(n) < n + (2c/(n-1)) \sum_{1 \text{ to } n-1} (i \log i)$
- $< n + (2c/(n-1)) \sum_{1 \text{ to } n-1} (i \log(n-1))$
- $= n + (2c/(n-1)) \log(n-1) \sum_{1 \text{ to } n-1} (i)$
- $= n + (2c/(n-1)) \log(n-1) n(n-1)/2$
- $= n + cn \log(n-1)$
- $\approx cn \log n - cn(1/n) + n$
- $> cn \log n !!$



# Cancel the “n”

$$\begin{aligned} T(n) &< n + (2c/(n-1)) \sum_{1 \text{ to } n-1} (i \log i) \\ &= n + (2c/(n-1)) [\sum_{1 \text{ to } n/2} (i \log i) + \sum_{n/2+1 \text{ to } n-1} (i \log i)] \\ &< n + (2c/(n-1)) [\sum_{1 \text{ to } n/2} (i \log (n/2)) + \sum_{n/2+1 \text{ to } n-1} (i \log n)] \\ &= n + (2c/(n-1)) [((\log n) - 1) \sum_{1 \text{ to } n/2} (i) + (\log n) \sum_{n/2+1 \text{ to } n-1} (i)] \\ &= \dots\dots \\ &= \dots\dots \\ &< c n \log n \end{aligned}$$

$$\begin{aligned}
T(n) &< n + (2c/(n-1)) \sum_{1 \text{ to } n-1} (i \log i) \\
&= n + (2c/(n-1)) [\sum_{1 \text{ to } n/2} (i \log i) + \sum_{n/2+1 \text{ to } n-1} (i \log i)] \\
&< n + (2c/(n-1)) [\sum_{1 \text{ to } n/2} (i \log (n/2)) + \sum_{n/2+1 \text{ to } n-1} (i \log n)] \\
&= n + (2c/(n-1)) [((\log n) - 1) \sum_{1 \text{ to } n/2} (i) + (\log n) \sum_{n/2+1 \text{ to } n-1} (i)] \\
&= n + (2c/(n-1)) [(-1) \sum_{1 \text{ to } n/2} (i) + (\log n) \sum_{1 \text{ to } n-1} (i)] \\
&= n + (2c/(n-1)) [(-1) \sum_{1 \text{ to } n/2} (i)] + (2c/(n-1)) [(\log n) \sum_{1 \text{ to } n-1} (i)] \\
&= n - (2c/(n-1)) \sum_{1 \text{ to } n/2} (i) + (2c/(n-1)) [(\log n) \sum_{1 \text{ to } n-1} (i)] \\
&= n - (2c/(n-1)) (n/2)(n/2+1)/2 + (2c/(n-1)) [(\log n) \sum_{1 \text{ to } n-1} (i)] \\
&= n - (1/4)cn(n+2)/(n-1) + cn \log n \\
&< n - (1/4)cn + cn \log n \\
&= cn \log n, \text{ choosing } c \geq 4.
\end{aligned}$$







# Multiplication of Large Integers

Consider the problem of multiplying two (large)  $n$ -digit integers represented by arrays of their digits such as:

$$A = 12345678901357986429 \quad B = 87654321284820912836$$

The grade-school algorithm:

$$\begin{array}{r} a_1 \ a_2 \ \dots \ a_n \\ b_1 \ b_2 \ \dots \ b_n \\ \hline (d_{10}) \ d_{11} d_{12} \ \dots \ d_{1n} \\ (d_{20}) \ d_{21} d_{22} \ \dots \ d_{2n} \\ \dots \ \dots \ \dots \ \dots \ \dots \ \dots \ \dots \\ \hline (d_{n0}) \ d_{n1} d_{n2} \ \dots \ d_{nn} \end{array}$$

Efficiency:  $n^2$  one-digit multiplications

# First Divide-and-Conquer Algorithm

A small example:  $A * B$  where  $A = 2135$  and  $B = 4014$

$A = (21 \cdot 10^2 + 35)$ ,  $B = (40 \cdot 10^2 + 14)$  Split into  $n/2$

So,  $A * B = (21 \cdot 10^2 + 35) * (40 \cdot 10^2 + 14)$  digit numbers

$$= 21 * 40 \cdot 10^4 + (21 * 14 + 35 * 40) \cdot 10^2 + 35 * 14$$

In general, if  $A = A_1A_2$  and  $B = B_1B_2$  (where  $A$  and  $B$  are  $n$ -digit,  $A_1, A_2, B_1, B_2$  are  $n/2$ -digit numbers),

$$A * B = A_1 * B_1 \cdot 10^n + (A_1 * B_2 + A_2 * B_1) \cdot 10^{n/2} + A_2 * B_2$$

Recurrence for the number of one-digit multiplications  $M(n)$ :

$$M(n) = 4M(n/2), \quad M(1) = 1$$

Solution:  $M(n) = n^2$

## Second Divide-and-Conquer Algorithm

$$A * B = A_1 * B_1 \cdot 10^n + (A_1 * B_2 + A_2 * B_1) \cdot 10^{n/2} + A_2 * B_2$$

The idea is to decrease the number of multiplications from 4 to 3:

$$(A_1 * B_2 + A_2 * B_1) = (A_1 + A_2) * (B_1 + B_2) - A_1 * B_1 - A_2 * B_2,$$

which requires only 3 multiplications at the expense of (4-1) extra add/sub (we already calculated  $A_1 * B_1$  and  $A_2 * B_2$  )

Recurrence for the number of multiplications  $M(n)$ :

$$M(n) = 3M(n/2), \quad M(1) = 1;$$

Use backward substitution:

$$\text{Solution: } M(n) = 3^{\log_2 n} = n^{\log_2 3} \approx n^{1.585}$$

$n$  needs to be quite large to run faster than classic version

# Strassen's Matrix Multiplication

Strassen observed [1969] that the product of two matrices can be computed as follows:

$$\begin{pmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{pmatrix} = \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix} * \begin{pmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{pmatrix}$$
$$= \begin{pmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{pmatrix}$$

# Strassen's Matrix Multiplication

$$\begin{pmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{pmatrix} = \begin{pmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{pmatrix}$$

$$M_1 = (A_{00} + A_{11}) * (B_{00} + B_{11})$$

$$M_2 = (A_{10} + A_{11}) * B_{00}$$

$$M_3 = A_{00} * (B_{01} - B_{11})$$

$$M_4 = A_{11} * (B_{10} - B_{00})$$

$$M_5 = (A_{00} + A_{01}) * B_{11}$$

$$M_6 = (A_{10} - A_{00}) * (B_{00} + B_{01})$$

$$M_7 = (A_{01} - A_{11}) * (B_{10} + B_{11})$$

Strassen's algorithm makes 7 multiplications and 18 additions/subtractions. How does this compare to brute-force algorithm? (8 multiplications + 4 additions)

# Strassen's Matrix Multiplication

We can divide A, B, and their product C into four  $n/2$  by  $n/2$  submatrices as follows:

$$\begin{pmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{pmatrix} = \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix} * \begin{pmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{pmatrix}$$

$n/2$



$$= \begin{pmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{pmatrix}$$

where submatrices can be computed recursively.

# Analysis of Strassen's Algorithm

- Let  $M(n)$  be #multiplication required.
- So,  $M(n)=7M(n/2)$ .
- By expansion (recursive method),
- $M(n) = 7^{\log n} = n^{\log 7} \approx n^{2.807}$
  
- Let  $A(n)$  be #addition required.
- $A(n)=7A(n/2)+18n^2$
- $A(n) = O(n^{\log 7}) \approx O(n^{2.807})$